

Cheat sheet: vocabulary types (Session 2)

One page for the types that replace raw pointers, sentinel values, out-parameters, and printf. Badge in brackets is the standard that introduced it.

Which type

You need	Use	Not
Read-only text parameter	<code>std::string_view</code> by value [17]	<code>const std::string&</code> , <code>const char*</code>
Read-only contiguous elements	<code>std::span<const T></code> by value [20]	<code>const T*</code> plus a length, <code>const std::vector<T>&</code>
A buffer to write into	<code>std::span<T></code> [20]	<code>T*</code> plus a length
A value you will keep	<code>T</code> by value, then <code>std::move</code> it	<code>const T&</code> plus a copy
Maybe one <code>T</code>	<code>std::optional<T></code> [17]	<code>T*</code> , sentinel values (-1, ""), <code>std::pair<T, bool></code>
A <code>T</code> or a reason there is none	<code>std::expected<T, E></code> [23], <code>[[nodiscard]]</code>	<code>optional</code> (loses the reason), <code>bool</code> plus out-parameter, exceptions for expected failures
Exactly one of a fixed set	<code>std::variant<A, B, C></code> [17]	tag plus union, <code>void*</code> , base class plus <code>dynamic_cast</code>
One of an open set, fixed operations	virtual base class	variant
Anything, checked at runtime	<code>std::any</code> [17]	<code>void*</code>
Someone else's <code>T</code> , maybe absent	<code>T*</code> non-owning	<code>optional<T&></code> (not in C++23), <code>shared_ptr</code>
Someone else's <code>T</code> , never absent	<code>T&</code>	<code>T*</code>
Owned, one owner	<code>std::unique_ptr<T></code> [11/14]	raw owning pointer, <code>shared_ptr</code> by default
Owned, shared for real	<code>std::shared_ptr<T></code>	anything you could express with <code>unique_ptr</code>
A line of text output	<code>std::format</code> [20] / <code>std::print</code> [23] with a <code>std::formatter</code>	<code>printf</code> , <code>iostream</code>

The one rule for views

`string_view`, `span`, and `mdspan` are a pointer and a length. They own nothing.

- Never store one unless you own what it points at and outlive it.
- Never return one into a local or a temporary (`std::string_view(s).substr(...)` of a local `s` dangles on return).
- `string_view` is not NUL-terminated: do not pass `.data()` to a C API; copy to a `std::string` first.
- `span<T>` converts to `span<const T>`; `span<T, N>` (static extent) is a compile-time size check at hardware boundaries; `std::as_bytes / as_writable_bytes` for byte views.
- Keep `const std::string&` only when the callee needs a `std::string` anyway (stores it, calls `c_str()`, or passes it on as a string).

optional and expected in one minute

Operation	optional<T>	expected<T, E>
Test	<code>if (o), o.has_value()</code>	<code>if (e), e.has_value()</code>
Read (checked)	<code>o.value()</code> throws <code>bad_optional_access</code>	<code>e.value()</code> throws <code>bad_expected_access<E></code>
Read (unchecked)	<code>*o, o->m</code>	<code>*e, e->m</code>
Fallback	<code>o.value_or(x)</code>	<code>e.value_or(x),</code> <code>e.error_or(err)</code>
Error	none	<code>e.error(),</code> <code>std::unexpected(err)</code> to construct
Map, can't fail	<code>o.transform(f)</code> [23]	<code>e.transform(f)</code>
Map, can fail	<code>o.and_then(f)</code> (<code>f</code> returns optional) [23]	<code>e.and_then(f)</code> (<code>f</code> returns expected)
Recover	<code>o.or_else(g)</code> [23]	<code>e.or_else(g),</code> <code>e.transform_error(h)</code>
Nothing to return	<code>std::nullopt</code>	<code>std::expected<void, E>{ }</code> on success

- `transform(&T::member)` does not compile: the pointer-to-member yields a reference and `optional<T&>` is ill-formed. Use a lambda.
- Neither type is `[[nodiscard]]` by itself. Put `[[nodiscard]]` on every function that returns one.
- Error type: a small enum class or a struct with a code and a message. Convert at API boundaries; do not propagate strings.
- `expected` for failures the caller is expected to handle (parse errors, lookups). Exceptions for failures nobody local can handle.

variant and visit

- `std::holds_alternative<T>(v), std::get<T>(v)` (throws `bad_variant_access`), `std::get_if<T>(&v)` (returns pointer or null), `v.index()`.
- Visitor pattern: template `<class... Fs> struct overloaded : Fs... { using Fs::operator()...; };` plus, in C++17 only, the deduction guide template `<class... Fs>`

overloaded(Fs...) -> overloaded<Fs...>; (C++20 deduces it). Then
 std::visit(overloaded{ [](A&){...}, [](B&){...} }, v).

- A visitor that misses an alternative fails to compile, unless another lambda accepts it by implicit conversion (float to a double lambda). Prefer exact-type lambdas or auto&& as a deliberate catch-all.
- Default-constructed variant holds the first alternative; use std::monostate first if none is default-constructible.
- valueless_by_exception() only after a throwing move or copy during assignment; most code can ignore it.

format and print

std::format(fmt, args...) returns a std::string [20]. std::print / std::println write it [23]. The spec goes after the colon: {:<10} left-align width 10, {:>8} right-align, {:08.3f} zero-pad width 8 precision 3, {:x} hex, {:#b} binary with prefix, {} for any type with a formatter. Positional: {0} {1} {0}.

From	To
printf("%d %s %.2f", i, s.c_str(), d)	print("{} {} {:.2f}", i, s, d)
printf("%-10s %5d", ...)	print("{:<10} {:>5}", ...)
std::cout << std::hex << x	print("{:x}", x) (no sticky state)
std::ostringstream	std::format
snprintf(buf, n, ...)	std::format_to_n(buf, n, ...); .out - buf is the length written, .size the length wanted

Your own type: specialize template <> struct std::formatter<MyType> with constexpr auto parse(auto& ctx) and auto format(const MyType& v, auto& ctx) const that returns std::format_to(ctx.out(), ...). Inherit from std::formatter<std::string_view> to reuse the standard spec. Ranges format as [a, b, c] from C++23 (GCC 15; libc++ 18).

clang-tidy modernize-use-std-print converts printf calls; modernize-use-std-format converts snprintf into format.

Library tour, by standard

- **C++17:** std::filesystem (path, exists, create_directories, directory_iterator); std::byte; to_chars / from_chars (no locale, no errno, no allocation); std::invoke, std::apply; std::clamp, std::gcd, std::lcm; map try_emplace, insert_or_assign, extract, merge; std::size, std::data, std::empty; transparent comparators std::less<> [14] let std::map<std::string, T> find by string_view (for unordered_map you also need a transparent hash and std::equal_to<>).
- **C++20:** std::bit_cast, <bit> (popcount, countl_zero, has_single_bit, rotl); std::source_location; std::erase / erase_if for every container; starts_with / ends_with; contains on associative containers; <chrono> calendars and time zones; std::numbers::pi; std::midpoint, std::lerp; std::to_array.

- **C++23:** `std::flat_map` / `flat_set` (sorted vectors behind a map interface; GCC 15, libc++ 20 / 21); `std::stacktrace` (GCC, link `-lstdc++exp`); `std::move_only_function` (libstdc++ only); `std::string::contains`; `std::byteswap`; `std::out_ptr` / `inout_ptr` for `T**` C APIs; `std::forward_like`; `resize_and_overwrite`; `std::mdspan` (libc++ 17, GCC 15).

Deprecated and removed

Deprecated in C++17 and still present: `std::iterator`, `<codecvt>` (removed in C++26 with `strstream`). Deprecated in C++17, removed in C++20: `std::result_of`, `std::uncaught_exception`, `<ciso646>`. Find them with `-Wdeprecated-declarations` (on by default on GCC and Clang).

Toolchain notes for this session

`std::expected` needs libstdc++ 12+ or libc++ 16+; Clang with libstdc++ lacks it before Clang 19 (concepts macro). `from_chars` for double needs libstdc++ 11+ or libc++ 20+. `std::print` needs GCC 14 or libc++ 17. Full table: [handouts/toolchain-support-matrix.md](#).